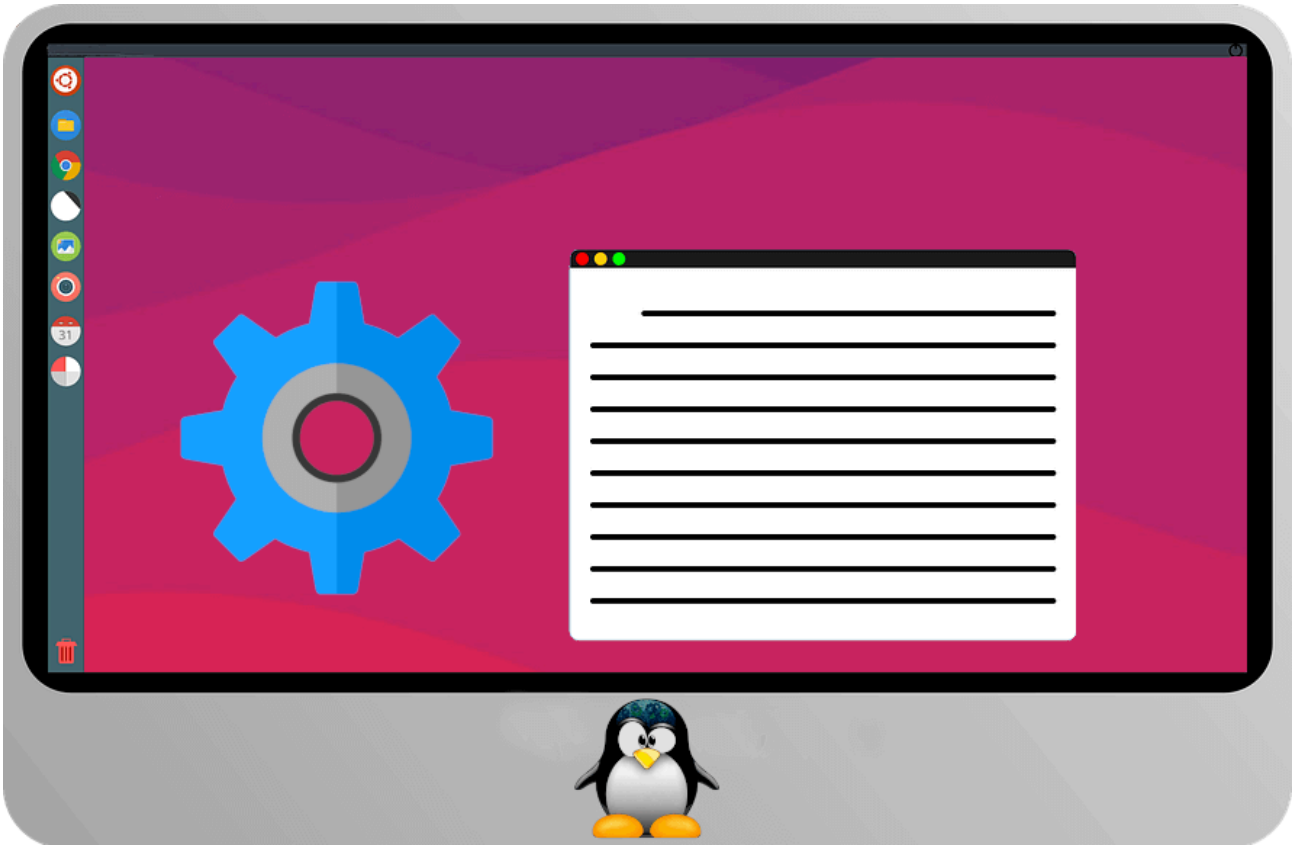
Advanced Ubuntu Configuration with Active Directory – Michael Waterman

michaelwaterman.nl/2022/10/07/advanced-ubuntu-configuration-with-active-directory

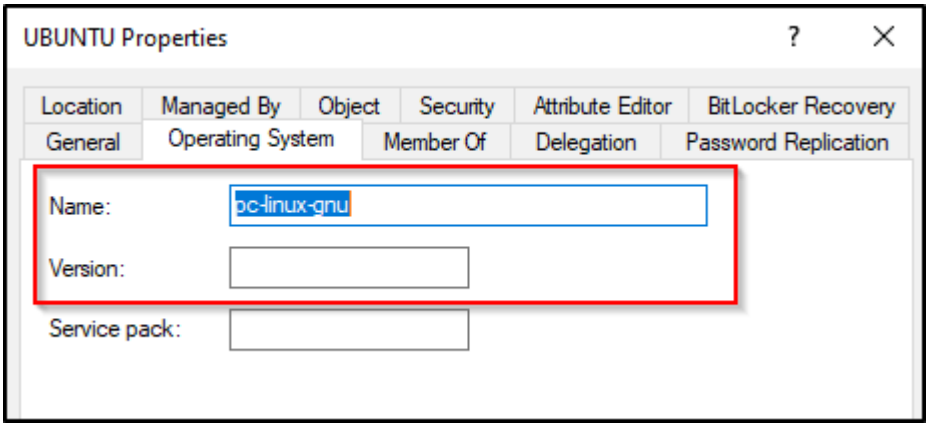
Michael Waterman October 7, 2022



In the previous blog post I wrote about how to join a Ubuntu 22.04 machine to a Microsoft Active Directory domain. In this follow up post I want to dive a little deeper into the configuration files, a bug I ran into during testing and setting some advanced security settings for access management. The latter is crazy easy actually, keep on reading.

Default configuration with realmd.conf

When a Ubuntu machine is joined to a Microsoft Active Directory domain, the join process depends on a number of predefined parameters. Things such as the way a home directory is constructed, how users will login and in general how the SSSD daemon will configure the machine and control the join process. One of the very visible examples is the name and version of the operating system as displayed on the computer object. By default the name property is set to "*pc-linux-gnu*" and the version info remains empty.



The way we can fix this is manually change the properties on the Active Directory Computer object, but that's way to much manual work. What if we could do this before the machine is joined to the domain? Well luckily we can by means of a pre-configuration file named "*realmd.conf*", located in the "*/etc/*" directory.

Hint! If you want to use the same machine as in the previous blog post, run the following command to leave the domain first:

```
sudo realm leave water.lab
```

Bash

Create the "realmd.conf" file and create the first entries:

```
sudo touch /etc/realmd.conf && sudo nano /etc/realmd.conf
```

Bash

paste the following text, save and close the file.

```
[active-directory]
default-client = sssd
os-name = Ubuntu Workstation
os-version = 22.04
```

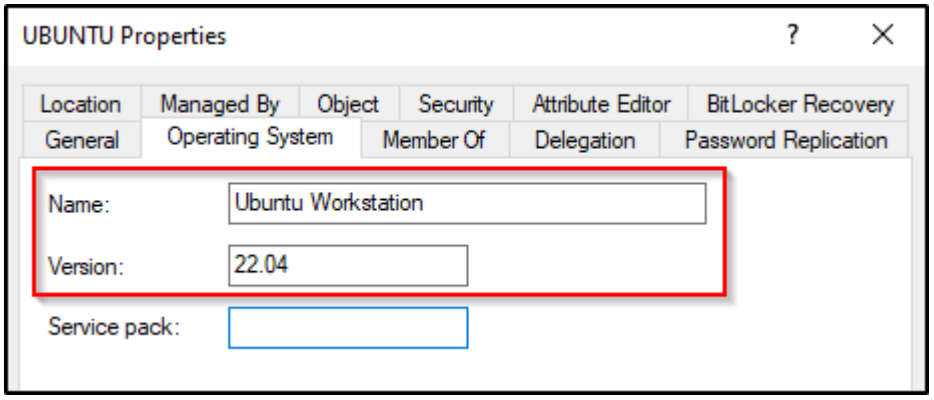
INI

Join the machine to the domain and see the magic happen!

```
sudo realm join --verbose --user=administrator water.lab
```

Bash

Open the computer object in Active Directory Users and Computers, browse to the "*Operating System*" tab and the field are now filled.



This is obviously just a small fraction on how you can influence the domain join process and the configuration of the SSSD daemon. The file itself uses a very simple construct of a key-value pair divided into sections, much like ini files which are often used on Windows based operating systems. I've listed the frequently used settings in the table below. As a means of convenience I've also added the related key in the "*sssd.conf*" file.

Section: [users]	Default value	realmd.conf property	Description
default-home	/home/%u@%d	fallback_homedir	Configures the layout of the home directory. For example, "*/home/%D/%U*" will create a directory with the domain name first and place a directory based on the name of the user inside of it.
default-shell	/bin/bash	default_shell	Sets the default shell for the user.

Section: [active-directory]	Default value	realmd.conf property	Description
default-client	*sssd* or *winbind*	[sssd] section	Sets the default client software package for joining and managing the domain activities.
os-name	pc-linux-gnu	None	Sets the Active Directory property "*OperatingSystem*" of the computer object.
os-version	None	None	Sets the Active Directory property "*OperatingSystemVersion*" of the computer object.

Section: [service]	Default value	realmd.conf property	Description
automatic-install	No	None	automatic installation of required and missing packages using “*package-kit*“. Note that “*package-kit*” itself needs to be installed first.

[realmname]	Default value	realmd.conf property	Description
automatic-id-mapping	On	ldap_id_mapping	leave this value set to default if you don't have the POSIX attributes set in Active Directory.
automatic-join	Off	None	Automatically joins a machine to active directory if a computer object already exists.
computer-ou	Off	None	Sets the location of the computer object in Active Directory, use DN notation. E.G. "*OU=Ubuntu, DC=water,DC=lab*"
computer-name	Off	None	Define the name of the computer object. Leave this value to default as it can lead to misconfigurations. Only set for in exceptional circumstances.
fully-qualified-names	On	use_fully_qualified_names	Determines if a user logon needs to be done with the UPN or just the username. Default is UPN.
manage-system	On	realmd_tags	Influences management from the domain. Recommended to leave this enabled, which is the default.
user-prinicpal	On	None	Creates a service principle name (SPN) on the computer object

Tip! To see the UPN being applied, use the following PowerShell command on a Domain joined Window device:

```
Get-ADComputer -filter * | Sort-Object -Property Name | Where Name -EQ "ubuntu" | Format-Table -Property Name, UserPrincipalName
```

PowerShell

The result should look like this:

Just as an example, here's my "*realmd.conf*" file.

```
[users]
default-home = /home/%D/%U
default-shell = /bin/bash

[active-directory]
default-client = sssd
os-name = Ubuntu Workstation
os-version = 22.04

[service]
automatic-install = no

[water.lab]
computer-ou=OU=linux,OU=End-Points,OU=Management,DC=water,DC=lab
fully-qualified-names = no
automatic-id-mapping = yes
user-principal = yes
manage-system = yes
```

INI

The thing about Kerberos

I learned a valuable lesson when experimenting with Linux integration with Microsoft Active Directory. One of them being that Linux is not Windows and vice versa. Both have their strengths and weaknesses. Microsoft on one hand obfuscates a lot of the underlying complexity by providing a UI to accomplish difficult tasks, that's 'one of the reasons Windows grew to the platform it is today. On the other hand Linux is extremely customizable and can run on practically anything, but you need knowledge on how things actually work. The latter is actually true for both operating systems once you dive a little deeper. While conceptually a lot of the services and application behave the same, the underlying way of accomplishing a task can be approached very differently. So just remember, both operating can do a lot of the same tings, but the way getting there can be a very different path.

This is particularly true for the subject of Kerberos authentication. On Windows, with the integration of Active Directory, things are portrait as simplistic, on Linux not so much, specially when you're starting on the topic. Once you're running into stuff not working and you really need to do a deep dive into the topic it can be overwhelming. I had the same experience during the creation of this series of blog posts.

After successfully joining the domain, I logged in with a domain user, checked the existence of the Kerberos TGT with "*klist*" and tried to open a SMB share with Nautilus on one of my servers. Much to my surprise I was greeted with a login prompt. Like what gives? Checking for the existence of a servicing ticket revealed that there was none, "that's

weird", I thought. Just to make sure everything was configured correctly the thought occurred to me that it could be because I didn't explicitly added a service principle name (SPN) for CIFS. No harm in setting those. So I opened up a command-prompt on Windows and entered:

```
setspn -a CIFS/lab-file lab-file
setspn -a CIFS/lab-file.water.lab lab-file
setspn -l lab-file
```

Bash

The latter revealed that the SPN properties where correctly set.

```
C:\Users\Administrator>setspn -l lab-file
Registered ServicePrincipalNames for CN=LAB-FILE,OU=Block NTLM,OU=Management,DC=water,DC=lab:
        CIFS/lab-file
        CIFS/lab-file.water.lab
        WSMAN/lab-file
        WSMAN/lab-file.water.lab
        TERMSRV/LAB-FILE
        TERMSRV/lab-file.water.lab
        RestrictedKrbHost/LAB-FILE
        HOST/LAB-FILE
        RestrictedKrbHost/lab-file.water.lab
        HOST/lab-file.water.lab
```

I tried it again, yet to no avail. Well, can't hurt having the SPN's in place so I just let them exist. Next was trying a different tools as I noticed that a lot of people mentioned that not every application on Linux was capable of using Kerberos for authentication, so Nautilus itself could even be a problem. One would expect however that an inbox application could handle this common scenario, but perhaps I was expecting too much.

Next tool I tried was "*smbclient*", a command-line tool on Linux to interact with SMB (CIFS) shares, see if that works. Apparently this tool isn't available by default on Ubuntu so I first had to install it.

```
sudo apt install smbclient
```

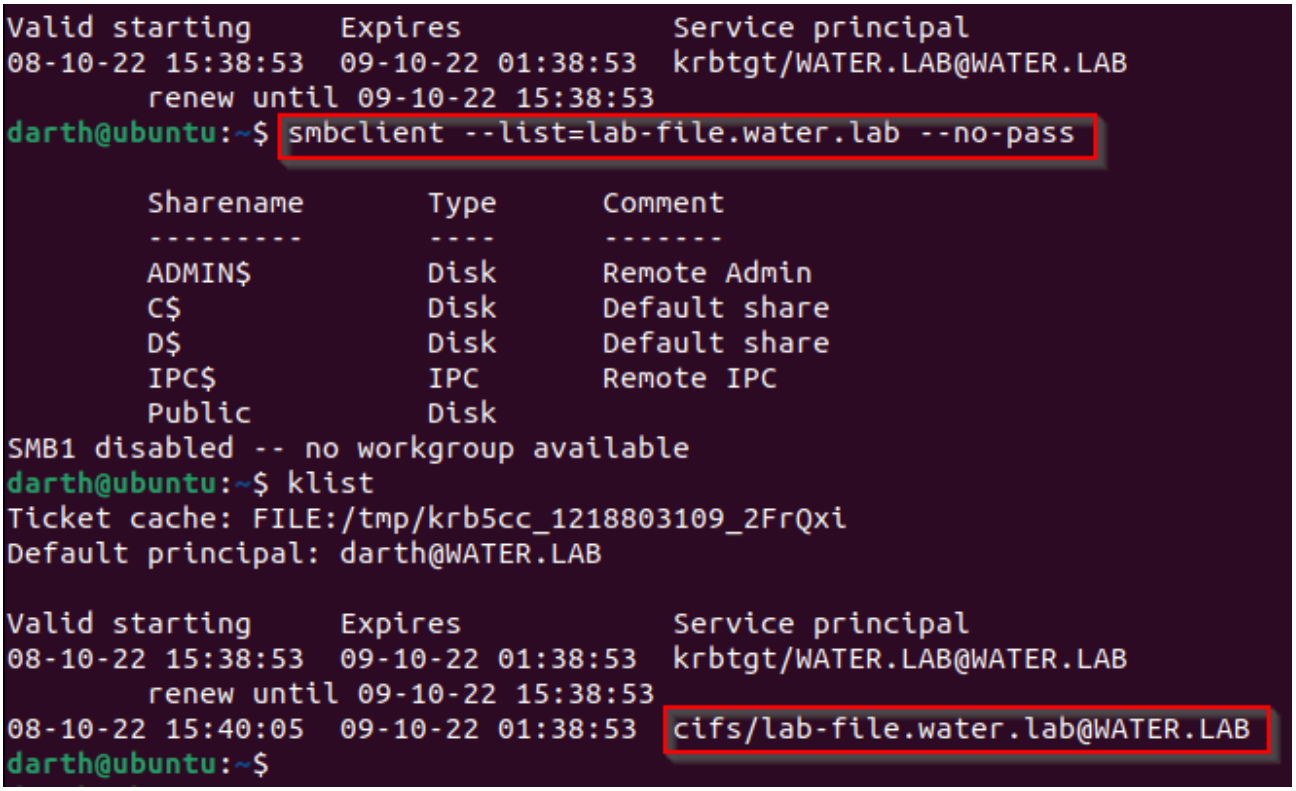
Bash

Next I tried to list the shares for the file server with the "*smbclient*" tool.

```
smbclient --list=lab-file.water.lab --no-pass
```

Bash

That had a satisfying effect! The shares where listed and a service ticket was created.



Just to see if I could also access one of the shares, I tried an alternative form of the command.

```
smbclient //lab-file.water.lab/Public --use-kerberos=required --use-krb5-ccache=CCACHE
```

Bash

As expected, this listed the content of the "*Public*" share. Please note that "*smbclient*" will always prompt for a password, so you need to explicitly provide it with either of the parameters I used in the previous two examples.

So up to this point there was no need to further investigate that Kerberos was not setup correctly, I've proven that it works. I had a hunch that it could be Nautilus itself. A lot of people on the Internet still claimed that Nautilus just wasn't capable of using Kerberos, but on the other hand a significant number of people claimed that it had previously worked, could be a bug. That eventually let me to the bug tracking for Ubuntu, specifically this one that described my exact problem:

Nautilus does not use a valid Kerberos ticket when accessing Samba share

I was surprised that this bug hasn't been fixed as it's been active as of early 2018, but luckily there's a workaround available. Follow these steps:

```
sudo nano /usr/lib/systemd/user/gvfs-daemon.service
```

Bash

Enter the following in the [services] section:

```
ExecStartPre=bash -c "for i in echo {1..20} ; do ps ax | grep -q \"^${USER}\b.*
[i]bus-daemon\" || sleep 0.1 ; done"
```

Bash

This should do the trick! An alternative could be using an alternative file explorer, like “Dolphin”, but that’s up to you.

```
apt install dolphin kio-extras
```

Bash

Configuring local logon rights

Now that our Ubuntu machine is part of the domain anyone from that domain can logon to the machine, perhaps that’s not something you want. I can imagine that you’re going to use a Ubuntu machine as an administrative box to reduce the attack vector and be different than anyone else. Which is a great idea, but you would need to configure that first. Lucky the tools are already available to make that happen. Listed here are a few examples:

Allow all users (the default)

```
sudo realm permit --all
```

Bash

Deny all users

```
sudo realm deny --all
```

Bash

Allow a single user

```
sudo realm permit darth@water.lab
```

Bash

Allow multiple users

```
sudo realm permit darth@water.lab luke@water.lab
```

Bash

Allow a group

```
sudo realm permit -g 'Domain Admins'
```

Bash

Deny a group

```
sudo realm permit --withdraw --groups 'Domain Admins'
```

Bash

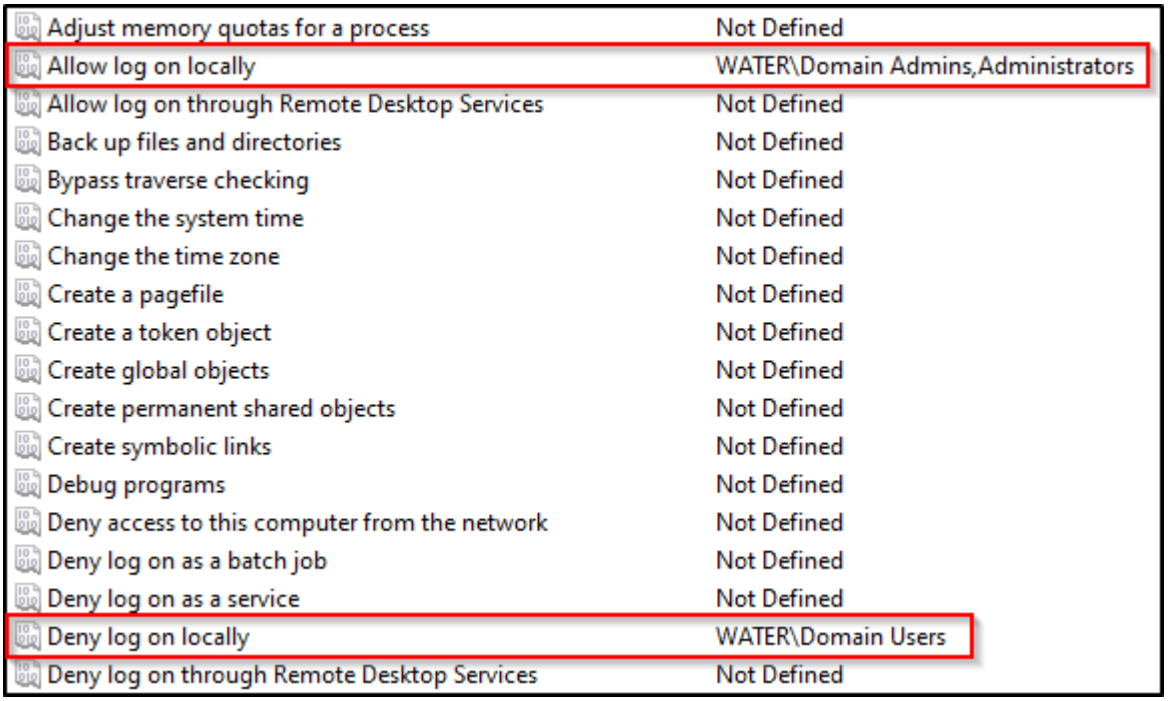
What about Group Policies?

With the release of Ubuntu 22.04, more centralized management from Active becomes available for Ubuntu systems, but that's a blog post for another time. Even in previous versions, a couple of security related settings can be centrally managed by utilizing Group Policies from the domain. Let's take our previous example of allowing and denying users and groups. Suppose we want to make our Ubuntu machine an admin workstation, we could do it manually like in the previous chapter, and that's fine for a single or a few machines but you want more centralized control. Group Policies to the rescue!

On a Windows machine, open up "*Group Policy Management*", create a new policy with the scope of management applied to the Ubuntu machine. Open the policy for editing and go to:

"*Computer Configuration – Policies – Windows Settings – Security Settings – Local Policies – User Right Assignment*"

- Allow logon locally: "Administrators", "Water\Domain Admins"
- Deny logon locally: "Water\Domain Users"



Reboot your machine or restart the sssd daemon. Try to logon with a regular user account, this will fail, however a subsequential logon with a domain admin account will be granted.

Controlling Group Policy processing

The processing of group policies can be configured in the "*/etc/sssd/sssd.conf*" file. Sometimes it could be beneficial to not have policies being processed for troubleshooting purposes, but it could be in violation with your companies policy. This is also another

reminder why you should avoid handing out sudo privileges to users as they could potentially block the required configuration, but that's another story. Enter the following commands to open the file, enter the correct settings and block gpo processing.

```
sudo nano /etc/sssd/sssd.conf
```

Bash
Enter the following under [domain\domainname] section:

```
ad_gpo_access_control = disabled
```

Bash
restart the sssd daemon or reboot and policies will no longer process. Potential other values for the key "*ad_gpo_access_control*" are:

Value	Description
disabled	Processing of Group Policies is disabled
enforcing	Processing of Group Policies is enabled and forcefully applied to the system
permissive	Processing of Group Policies is being audited. Deny messages will be logged.

According to my findings, but I didn't test them all, the following Group Policy Security settings will by default be processed:

- Allow log on locally
- Deny log on locally
- Allow log on through Remote Desktop Services
- Deny log on through Remote Desktop Services
- Access this computer from the network
- Deny access to this computer from the network
- Allow log on as a batch job
- Deny log on as a batch job
- Allow log on as a service
- Deny log on as a service

Troubleshooting

Sometimes things break, don't work from the start or there's a misconfiguration somewhere. Where do you start? That's where you need to have a little knowledge on how to troubleshoot the technology. Below are a few tips and tricks that you can use to troubleshoot your configuration when connecting to Active Directory.

Configuration Files

Relevant configuration files for troubleshooting.

Filename	Description
/etc/krb5.conf	Kerberos configuration file. Usually this can be left to default.
/etc/sssd/sssd.conf	Main configuration file for realm configuration.
/etc/realmd.conf	Initial configuration file for sssd.
/etc/nsswitch.conf	Service configuration for hostnames, password processing etc.

Services

SSSD status

```
systemctl status sssd
```

Bash

Networking

Get the local hostname

```
hostname -f
```

Bash
Note! This should return the fqdn of the host.

Get the network status

```
resolvectl status
```

Bash
Basic name resolving

```
dig -t a lab-dc01.water.lab
```

Bash
Basic name resolving with specific DNS server

```
dig -t a lab-dc01.water.lab @192.168.66.2
```

Bash
Query domain SRV records

```
dig -t SRV _ldap._tcp.ad.water.lab
```

Bash

Disable Reverse name lookup

```
sudo nano /etc/krb5.conf
```

Bash
Add the following:

INI

```
[libdefaults]
rdns = false
```

INI

SSSD Configuration

List sssd configuration

```
realm list
```

Bash
List sssd status

```
realm list | grep configured
```

Bash
The output should be: "*configured: kerberos-member*"

Check the configuration of SSSD

```
sudo sssctl config-check
```

Bash

Information Retrieval

Retrieve user information from Active Directory

```
getent passwd darth@water.lab
```

Bash

Get user groups

```
Option 1:
groups darth@water.lab

Option 2:
id darth@water.lab
```

Bash
Advanced information

```
sudo sssctl user-checks darth
```

Bash

Show discovered Active Directory Servers

```
sudo sssctl domain-status water.lab --servers
```

Bash

Show active Active Directory servers

```
sudo sssctl domain-status water.lab --active-server
```

Bash

Authentication

Lists a users kerberos tickets

```
klist
```

Bash

Delete kerberos tickets

```
kdestroy
```

Bash

Manually get a kerberos ticket

```
kinit darth@water.lab
```

Bash

Manually test a login

```
sudo login
```

Bash

Show cached user inforation

```
sudo sssctl user-show darth@water.lab
```

Bash

Expire the sssd cache

```
sudo sssctl cache-remove
```

Bash

Debugging

Enable debug mode

```
sudo sssctl debug-level [1-9]
```

Bash

Hint! You can also set the debug level directly in the "*/etc/sssd/sssd.conf*" file. Log information will end up in the "*/var/log/sssd/*.log*" files.

Determine the session to troubleshoot

```
sudo sssctl analyze request list -v
```

Bash

Note! for the analyze to work, the debug level should be set to 7 or higher.

Analyze a specific session

```
sssctl analyze request show 949
```

Bash

Live view of system events

```
sudo tail -f /var/log/syslog
```

Bash

References

[SSSD and Active Directory | Ubuntu]

[Manually Connecting an SSSD Client to an Active Directory Domain – Red Hat Customer Portal]

[Group Policy Object Access Control]

[Log Analyzer]

[Troubleshooting Basics – sssd.io]

This finishes up this post for today. Next up is managing sudoers from Active Directory.